

รายงานปฏิบัติการที่ 10

Spring WebFlux และ WebClient

ระบบ REST API จัดการสินค้าแบบ Non-blocking Reactive Programming

ผู้จัดทำ	นายปวิศรภ์ ประมวล
รหัสนักศึกษา	673380278-9
Section	1
รายวิชา	CP353002 Principles of Software Design
เทคโนโลยี	Spring Boot 3.3.0, Java 17, WebFlux, Reactor Netty, WebClient

1. Reactive Programming และ Non-blocking I/O

Reactive Programming เป็นแนวทางประมวลผลข้อมูลในรูป stream แบบ asynchronous โดย publisher ส่งสัญญาณ onNext, onComplete หรือ onError ไปยัง subscriber การทำงานจึงประกอบ operator ต่อกันได้โดยไม่ต้องให้ thread รอผลลัพธ์ I/O

Spring WebFlux เป็น reactive web stack ที่รองรับ Reactive Streams และทำงานแบบ non-blocking บน Reactor Netty ในโปรเจกต์นี้ controller คำนวณ Mono/Flux ให้ framework เป็นผู้ subscribe และเขียน HTTP response

Blocking เทียบกับ Reactive

ประเด็น	Blocking	Reactive / Non-blocking
การรอ I/O	Thread รอจนงานเสร็จ	คืน publisher แล้วใช้ callback/signal
รูปแบบข้อมูล	คืนค่าโดยตรง	คืน Mono หรือ Flux
การรองรับโหลด	มักใช้หนึ่ง thread ต่อ request	event-loop รองรับหลาย request ด้วย thread จำนวนน้อย
ตัวอย่าง client	RestTemplate หรือ blocking call	WebClient พร้อม operator chain

Mono และ Flux

Mono<T> ส่งข้อมูลได้ 0 หรือ 1 ค่า เหมาะกับ findById, save, price และ Mono<Void> สำหรับ delete ส่วน Flux<T> ส่งข้อมูลได้ 0 ถึง N ค่า เหมาะกับ findAll และการค้นหาตาม category

```
Mono<Product> one = repository.findById("1");  
Flux<Product> many = repository.findAll();
```

2. สถาปัตยกรรมและ Operators

ระบบแยกหน้าที่ตาม Controller - Service - Repository เพื่อให้แต่ละชั้นมีความรับผิดชอบชัดเจน และใช้ constructor injection ตามหลัก Dependency Inversion

HTTP Request

- > ProductController (Mono/Flux)
- > ProductService (business operators)
- > ProductRepository (ConcurrentHashMap)
- > Reactive signal -> HTTP Response

Operators ที่ใช้

map	ProductService	สร้าง ID และแปลง Product เป็น discountedPrice
filter	ProductRepository	กรองสินค้าที่ category ตรงกัน
flatMap	ProductService / tests	เชื่อม publisher สำหรับ save และ delete
switchIfEmpty	ProductService	เปลี่ยน empty publisher เป็น HTTP 404
defaultIfEmpty	WebClient consumer	กำหนดค่าทดแทนเมื่อ Mono ไม่มีข้อมูล

WebClient method chain

get/post/delete เลือก HTTP method, uri กำหนด endpoint, retrieve เริ่มอ่าน response และ bodyToMono/bodyToFlux แปลง body เป็น reactive publisher จากนั้น caller chain operator ต่อได้ทันที

```
productWebClient.getProductById("1")
    .map(Product::getName)
    .defaultIfEmpty("Product not found")
    .subscribe(System.out::println);
```

subscribe แสดงเฉพาะขอบระบบฝั่ง consumer ส่วน production controller/service ไม่ subscribe หรือ block เอง

3. REST API ที่พัฒนา

Metho d	Endpoint	Return	ผลลัพธ์
GET	/products	Flux<Product>	สินค้าทั้งหมด
GET	/products/{id}	Mono<Product>	สินค้าตาม ID หรือ 404
POST	/products	Mono<Product>	สร้างสินค้าและ UUID เมื่อไม่มี ID
DELETE	/products/{id}	Mono<Void>	ลบสำเร็จคืน 204
GET	/products/category/{category}	Flux<Product>	กรอง category
GET	/products/{id}/price	Mono<Double>	ราคาหลังส่วนลด

กฎส่วนลด

NONE คืนราคาเต็ม, MEMBER ลด 10% และ SEASONAL ลด 20% โดย Product.getDiscountedPrice() เป็นผู้คำนวณ

4. หลักฐานการเรียก API จริง

LAB 10 - SPRING WEBFLUX API EVIDENCE

GET /products

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

GET http://localhost:8080/products

RESPONSE - HTTP 200

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (นายปวิศษ์ ประมวล 673380278-9 SEC 1)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
```

นายปวิศษ์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 1: GET /products - Flux คินลินค้าตัวอย่าง

LAB 10 - SPRING WEBFLUX API EVIDENCE

GET /products/1

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

GET http://localhost:8080/products/1

RESPONSE - HTTP 200

```
{
  "id": "1",
  "name": "iPhone 15 Pro (นายปวิศษ์ ประมวล 673380278-9 SEC 1)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900.0,
  "discountType": "MEMBER",
  "discountedPrice": 35910.0
}
```

นายปวิศษ์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 2: GET /products/1 - Mono พร้อมชื่อและรหัสนักศึกษา

5. หลักฐานการเรียก API จริง

LAB 10 - SPRING WEBFLUX API EVIDENCE

POST /products

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

POST http://localhost:8080/products

REQUEST BODY

```
{
  "id": "evidence-1",
  "name": "Mechanical Keyboard",
  "category": "Accessories",
  "brand": "Keychron",
  "stock": 10,
  "price": 3500.0,
  "discountType": "MEMBER"
}
```

RESPONSE - HTTP 200

```
{
  "id": "evidence-1",
  "name": "Mechanical Keyboard",
  "category": "Accessories",
  "brand": "Keychron",

```

นายปวิศช์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 3: POST /products - สร้างสินค้าและคืนข้อมูลที่บันทึก

LAB 10 - SPRING WEBFLUX API EVIDENCE

GET /products/category/Accessories

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

GET http://localhost:8080/products/category/Accessories

RESPONSE - HTTP 200

```
[
  {
    "id": "evidence-1",
    "name": "Mechanical Keyboard",
    "category": "Accessories",
    "brand": "Keychron",
    "stock": 10,
    "price": 3500.0,
    "discountType": "MEMBER",
    "discountedPrice": 3150.0
  }
]
```

นายปวิศช์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 4: GET /products/category/Accessories - filter ตาม category

6. หลักฐานการเรียก API จริง

LAB 10 - SPRING WEBFLUX API EVIDENCE

GET /products/evidence-1/price

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

GET http://localhost:8080/products/evidence-1/price

RESPONSE - HTTP 200

3150.0

นายปวิศช์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 5: GET /products/evidence-1/price - MEMBER 3500 ลดเหลือ 3150

LAB 10 - SPRING WEBFLUX API EVIDENCE

DELETE /products/evidence-1

Captured from the running Reactor Netty server - 2026-09-17 10:11 ICT

REQUEST

DELETE http://localhost:8080/products/evidence-1

RESPONSE - HTTP 204

<empty response body>

นายปวิศช์ ประมวล | 673380278-9 | Sec 1

ภาพที่ 6: DELETE /products/evidence-1 - สำเร็จด้วย HTTP 204

7. โค้ด Reactive แต่ละชั้น

Repository

defer ทำให้การอ่านเกิดเมื่อมี subscriber, justOrEmpty รองรับการไม่พบข้อมูล และ filter ใช้กับ Flux โดยไม่ต้องแปลงเป็น collection แบบ blocking

```
public Mono<Product> findById(String id) {
    return Mono.defer(() -> Mono.justOrEmpty(store.get(id)));
}

public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(product -> category.equalsIgnoreCase(product.getCategory()));
}
```

Service

map จัดเตรียม ID, flatMap ส่งต่อไปยัง reactive repository และ switchIfEmpty คืน 404 เมื่อไม่พบสินค้า

```
public Mono<Product> save(Product product) {
    return Mono.just(product)
        .map(item -> {
            if (item.getId() == null || item.getId().isBlank())
                item.setId(UUID.randomUUID().toString());
            return item;
        })
        .flatMap(repository::save);
}

public Mono<Double> getDiscountedPrice(String id) {
    return getById(id).map(Product::getDiscountedPrice);
}
```

8. Controller และ WebClient

Controller

Controller คำนวณ Mono/Flux โดยตรง Spring WebFlux จึงจัดการ subscription และ response lifecycle ให้ทั้งหมด

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}
```

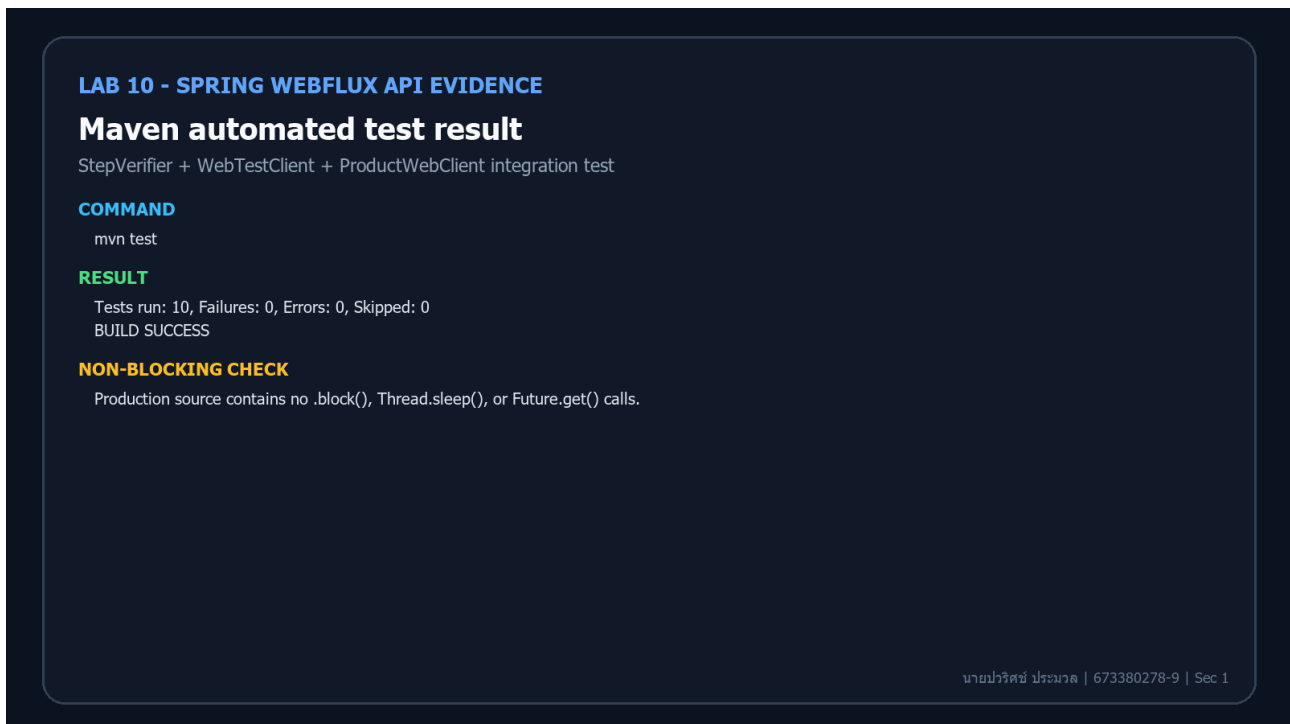
WebClient

WebClient ใช้ fluent API และ bodyToMono/bodyToFlux เพื่อคืน publisher ให้ caller ประกอบ operator ต่อโดยไม่ใช้ block

```
public Flux<Product> getAllProducts() {
    return client.get()
        .uri("/products")
        .retrieve()
        .bodyToFlux(Product.class);
}

public Mono<Product> createProduct(Product product) {
    return client.post()
        .uri("/products")
        .bodyValue(product)
        .retrieve()
        .bodyToMono(Product.class);
}
```

9. การทดสอบและผลลัพธ์



ภาพที่ 7: ผล Maven tests จาก StepVerifier, WebTestClient และ WebClient integration

ชุดทดสอบทั้งหมด 10 tests ผ่านโดยไม่มี failure หรือ error ครอบคลุม repository, service, ส่วนลดทั้งสามแบบ, HTTP endpoints และ ProductWebClient ที่เรียก Reactor Netty server จริงบน random port

ตรวจสอบ source แล้วไม่พบ .block(), blockFirst(), blockLast(), Thread.sleep() หรือ Future.get() ใน production code จึงไม่มีจุดที่ทำให้ endpoint รอผลแบบ blocking

สรุป

ระบบสามารถสร้าง ดู ค้นหาตาม category คำนวณราคาหลังส่วนลด และลบสินค้าได้ด้วย Mono/Flux พร้อม WebClient ที่ chain operators ต่อได้ทันทีตามข้อกำหนด

10. แหล่งอ้างอิง

1. Spring Framework Reference - Spring WebFlux

<https://docs.spring.io/spring-framework/reference/web/webflux.html>

2. Spring Framework Reference - WebClient

<https://docs.spring.io/spring-framework/reference/web/webflux-webclient.html>

3. Project Reactor Reference Guide - Getting Started

<https://projectreactor.io/docs/core/release/reference/gettingStarted.html>

4. Project Reactor API - Mono และ Flux

<https://projectreactor.io/docs/core/release/api/reactor/core/publisher/Mono.html>

<https://projectreactor.io/docs/core/release/api/reactor/core/publisher/Flux.html>

Repository

<https://github.com/PavaritPramual/posd-lab10-WebClient>